



Solution Design Document

ACDC - Card Fields

Installments (IC2B) · 3DS Risk Initiated · Vault · Fraudnet · STC

Document type	Solution Design Document (SDD)
Solution	Credit and debit card processing in a web checkout using PayPal Advanced Credit and Debit Card (ACDC)
Technical components	PayPal JavaScript SDK · Card Fields · Vault v3 · Calculated Financing Options · Risk Transaction Contexts (STC) · Fraudnet
Functional capabilities	New-card payment · Tokenization (Vault-with-Purchase) · Saved-card payment · Installments (IC2B) · 3DS Risk Initiated · Risk pre-evaluation (Fraudnet + STC)
REST APIs involved	/v1/oauth2/token · /v2/checkout/orders · /v3/vault/payment-tokens · /v1/credit/calculated-financing-options · /v1/risk/transaction-contexts
Conventions	REST → snake_case · JavaScript SDK → camelCase
Audience	Solution Architecture · Integration Engineering · Product · e-commerce · Risk · PCI Compliance teams

Table of Contents

Table of Contents	2
1. Executive Summary	6
2. Business Context and Drivers	7
2.1 Business Drivers	7
2.2 Regulatory and Compliance Drivers	7
3. Solution Scope	8
3.1 In-Scope Capabilities	8
3.2 Out-of-Scope Capabilities	8
4. Stakeholders and Audience	10
5. Glossary of Terms and Acronyms	11
6. Solution Overview	14
6.1 Solution Components	14
6.2 Correlation Identifier: the CMID	15
7. Functional Requirements	16
8. Non-Functional Requirements	17
9. Solution Architecture	19
9.1 Logical Diagram	19
9.2 Non-Negotiable Architecture Rules	19
9.3 Security Boundary by Asset	19
9.4 Architectural Pattern: Backend-for-Frontend (BFF) over PayPal	20
10. Prerequisites and Environment Configuration	21
10.1 Required Commercial Enablements	21
10.2 Backend Credentials and Configuration	21
10.3 Environment Requirements	21
11. OAuth2 Authentication and id_token Issuance	23
11.1 Request	23
11.2 Response (Relevant Fields)	23
11.3 access_token Cache	23
11.4 Endpoint Exposed to the Frontend	24
11.5 Reference Implementation (Node.js)	24
12. Client Metadata ID (CMID) Generation	25
12.1 Lifecycle Rules	25
12.2 Reference Implementation	25

12.3 Validation	26
13. Fraudnet Integration	27
13.1 Composition	27
13.1.1 Configuration Script (JSON)	27
13.1.2 Library Script	27
13.2 Dynamic Implementation	28
13.3 Operational Rules	28
14. Set Transaction Context (STC)	29
14.1 Endpoint	29
14.2 Body — Generic Retail Set	29
14.3 Field Reference	29
14.4 Response Handling — Non-Blocking Behavior	30
14.5 Reference Implementation	31
14.5.1 Backend (Node.js)	31
14.5.2 Frontend	31
15. Loading the PayPal JavaScript SDK	32
15.1 Building the SDK URL	32
16. Card Fields HTML Structure and Styles	33
16.1 DOM Containers	33
16.2 Rendering	33
16.3 Layer 1 — Container CSS	33
16.4 Layer 2 — SDK styleObject	33
17. Order Creation and Capture	35
17.1 Mandatory HTTP Headers	35
17.2 Create Order — Complete Payload (New Card with Vault)	35
17.3 Rationale for Each Block	36
17.4 Breakdown Validation Rules	37
17.5 Create Order with Saved Card (Token)	37
17.6 Propagating the CMID to the Backend Header	37
18. Installments — MSI and IC2B	39
18.1 New Card — SDK Callbacks	39
18.2 Submit with Installments	40
18.3 Saved Card — calculated-financing-options	40
18.4 Create Order with Token + Installments (REST uses snake_case)	40

19. Vault — Card Tokenization	42
19.1 customer.id — Stable Buyer Identifier	42
19.2 Vault-with-Purchase	42
19.3 Listing, Charging, and Deleting Tokens	42
19.4 Test Cards for Vault (Sandbox)	43
20. 3DS Risk Initiated and liabilityShift	44
20.1 Modes	44
20.2 Mandatory Logic in onApprove	44
20.3 Decision Table by liabilityShift Value	44
21. End-to-End Orchestration	46
21.1 Moment 1 — Checkout Initialization (Once per Session)	46
21.2 Moment 2 — Before Each Create Order (New or Saved Card)	46
21.3 Moment 3 — Capture Decision (in onApprove)	46
21.4 Moment 4 — Capture	46
21.5 Mnemonic Rule	47
22. Specialized Use Case — BOPIS	48
22.1 full_name Convention	48
23. Integration Points (Endpoint Map)	49
24. Security Considerations	50
24.1 Credential Custody (NFR-02)	50
24.2 PCI Isolation / CSP / Logging	50
25. Operational Considerations	51
25.1 Metrics and Observability	51
25.2 Fault Tolerance and Promotion to Live	51
26. Testing Strategy and Sandbox Cards	53
26.1 Common Sandbox Rules	53
26.2 Cards for Vault and Tokenization	53
26.3 Cards for 3DS Risk Initiated	53
26.3.1 Frictionless Flows (No User Modal)	53
26.3.2 Step-Up Flows (With User Modal)	54
26.4 Value Dictionary	55
26.4.1 liabilityShift (in onApprove)	55
26.4.2 authentication_status	55
26.4.3 enrollment_status	55

26.5 Minimum Test Cases (TC-01 to TC-14)	56
27. REST vs SDK Naming Conventions	57
28. Assumptions, Dependencies, and Constraints.....	58
28.1 Assumptions.....	58
28.2 External Dependencies.....	58
28.3 Constraints	59
29. Risks and Mitigations.....	60
30. Acceptance Criteria and Pre-Production Checklist	62
Appendix A — Troubleshooting Common Errors.....	64
Appendix B — Limitations and Future Work.....	66
B.1 Known Limitations	66
B.2 Suggested Future Work	66

Part I — Solution Context

1. Executive Summary

This Solution Design Document defines the integration of PayPal Advanced Credit and Debit Card (ACDC) into a web checkout experience. The solution enables credit and debit card payments directly on the merchant site through the Card Fields component of the PayPal JavaScript SDK, integrated with five additional PayPal capabilities: tokenization (Vault), installment payments (Installments — MSI and IC2B), 3-D Secure authentication in Risk Initiated mode, and two risk evaluation services (Fraudnet and Set Transaction Context).

The solution is intended for merchants that require:

- Reduced PCI DSS scope, maintaining SAQ A by preventing PAN and CVV from touching their infrastructure.
- Optimized conversion through native on-site payment collection, without redirecting to PayPal Checkout.
- Chargeback protection through Chargeback Protection (CBP) and 3DS Risk Initiated authentication.
- Buyer card reuse in subsequent purchases through opaque Vault tokens.
- Installment plans with or without buyer interest in eligible markets.
- Pre-transaction risk evaluation through device fingerprinting (Fraudnet) and buyer context (STC).

This document describes the solution end to end: business context, requirements, architecture, detailed component design, orchestration, security, operations, testing, risks, and acceptance criteria.

2. Business Context and Drivers

2.1 Business Drivers

Driver	Description
Conversion and user experience	Keeping the buyer on the merchant site during payment reduces friction compared with a redirected checkout. Card Fields allows the brand, styling, and navigation to remain under the merchant's control.
Reduced PCI DSS costs	PayPal-hosted iframes isolate PAN and CVV from the merchant environment, enabling PCI compliance under the simplified SAQ A questionnaire instead of SAQ D.
Fraud and chargeback protection	The combination of 3DS Risk Initiated + Chargeback Protection shifts liability for fraudulent chargebacks to the issuing bank when 3DS succeeds.
Higher average order value through Installments	Interest-free Installments (cost absorbed by the merchant) and IC2B (Installments Cost To Buyer, cost paid by the buyer) in eligible markets can increase conversion rates and average order value.
Recurrence and retention	Vault enables charging saved cards without requiring buyers to re-enter their details, reducing friction in subsequent purchases and enabling subscription and one-click checkout use cases.
Enriched risk evaluation	Fraudnet and STC feed PayPal's risk engine with browser telemetry and buyer context, improving approval rates for good transactions and reducing false positives.

2.2 Regulatory and Compliance Drivers

Driver	Description
PCI DSS	The standard requires strict controls for handling card data. Card Fields delegates that handling to PayPal.
PSD2 / SCA (Strong Customer Authentication) in Europe, and regional equivalents	3DS Risk Initiated provides strengthened authentication when required by the risk engine or issuing bank.
Local consumer protection regulations	Explicit consent to save the card (Vault-with-Purchase) and transparency around the financing fee (IC2B) must be reflected in the UI according to the target market's regulations.

3. Solution Scope

3.1 In-Scope Capabilities

Capability	Description
New-card payment	Card data capture through Card Fields (PayPal iframes) and processing through Create Order + Capture Order.
Tokenization (Vault-with-Purchase)	Optional card saving after the first successful purchase is completed.
Token listing and deletion	Management of a buyer's saved payment methods.
Saved-card payment (token)	Reuse of a PAYMENT_METHOD_TOKEN to create orders without re-entering card data.
Installments (interest-free)	Installment purchases with no interest for the buyer, where the cost is absorbed by the merchant.
Installments — IC2B (Cost to buyer)	Installment purchases where the financing cost is paid by the buyer.
3DS Risk Initiated	3-D Secure authentication decided by PayPal's risk engine; the merchant does not send 3DS instructions and only validates liabilityShift in onApprove.
Fraudnet	Device and browser telemetry collection on the client.
Set Transaction Context (STC)	Sending buyer context to the risk engine before each Create Order.
BOPIS	Shipping payload variant for pickup at a physical store.

3.2 Out-of-Scope Capabilities

Capability	Reason for Exclusion
Payment with PayPal account or PayPal branded button	The solution focuses exclusively on card payment through Card Fields.
Alternative Payment Methods (APMs): Pay Later, BNPL, Venmo, local wallets	Not required by the ACDC scope. They may be added in a complementary solution.
3DS Merchant Initiated	Applies to accounts with Fraud Protection (FP); this SDD describes the Risk Initiated flow, exclusive to accounts with Chargeback Protection (CBP).

Capability	Reason for Exclusion
Subscriptions and recurring plans orchestrated by PayPal Billing Plans	Recurring payment through a Vault token is covered; PayPal Billing-managed subscriptions are a different solution.
Asynchronous event webhooks (refunds, disputes, settlement)	Require a complementary back-office SDD.
Full or partial refunds	Back-office operation not covered by this SDD.
Accounting reconciliation and financial reporting	Outside the checkout scope.

4. Stakeholders and Audience

Role	Responsibility Regarding This SDD
Solution Architect	Validates the architecture, ensures consistency with the merchant portfolio, and approves the document.
Product Manager (e-commerce)	Validates that the functional scope covers business requirements and the desired checkout experience.
Integration Engineer (PayPal)	Supports the merchant during implementation, validates commercial configurations (CBP, Vault, Installments, STC), and provides industry packs when applicable.
Integration Engineer / Tech Lead (merchant)	Leads the technical implementation and owns the frontend and backend code.
Merchant development team	Implements and maintains the integration.
Risk and Anti-Fraud team (merchant)	Validates the Fraudnet and STC configuration, defines the additional_data field set, and monitors fraud metrics.
PCI Compliance team (merchant)	Verifies that the integration maintains SAQ A scope and approves the security configuration.
QA team (merchant)	Executes the Sandbox test plan before promotion to Live.
Operations (merchant)	Monitors checkout in production and manages alerts and incidents.

5. Glossary of Terms and Acronyms

Term	Definition
ACDC	Advanced Credit and Debit Card. PayPal product for processing credit and debit cards directly on the merchant site through the JavaScript SDK with Card Fields.
Card Fields	SDK component that renders card fields as <iframe> elements hosted by the PayPal domain. The PAN and CVV never reach the merchant server, which significantly reduces PCI DSS scope.
Order	REST resource (/v2/checkout/orders) representing the payment intent: amount, breakdown, items, buyer, and payment method. Identified by order_id.
Capture	Operation that performs the actual charge against the order's payment method (POST /v2/checkout/orders/{id}/capture). Funds do not move until capture occurs.
intent	Order field that indicates CAPTURE (charge on approval) or AUTHORIZE (only hold funds for later capture). The typical e-commerce usage is CAPTURE.
Vault	PayPal tokenization service. Securely stores the payment method and returns an opaque PAYMENT_METHOD_TOKEN that the merchant uses to charge future transactions.
PAYMENT_METHOD_TOKEN	Opaque identifier for a payment method saved in Vault. It does not contain sensitive card data.
customer.id	Stable buyer identifier in the merchant platform. Links Vault tokens to the same user. It must be deterministic across sessions.
MSI	Meses Sin Intereses / interest-free months. The financing cost is absorbed by the merchant. Identified by total_consumer_fee.value === "0.00".
IC2B	Installments Cost To Buyer. The financing cost is paid by the buyer. Identified by total_consumer_fee.value > "0.00". Requires fee_reference_id in the submit call.
3-D Secure (3DS)	Buyer authentication protocol with the issuing bank (Verified by Visa, Mastercard SecureCode, Amex SafeKey). When successful, it shifts fraud liability to the issuer.
3DS Risk Initiated	Mode in which PayPal's risk engine decides whether to trigger 3DS. The merchant does not send 3DS instructions; it only validates the result in onApprove through liabilityShift. Requires Chargeback Protection (CBP).

Term	Definition
3DS Merchant Initiated	Mode in which the merchant explicitly decides when to force 3DS through <code>payment_source.card.attributes.verification</code> . Requires Fraud Protection (FP). Not in scope for this SDD.
liabilityShift	<code>onApprove</code> callback field indicating the 3DS result. Typical values: undefined (no challenge), "POSSIBLE" (successful 3DS, liability shifted to issuer), "N" (failed), "U" (unavailable).
access_token	OAuth2 Bearer token used by the server to authenticate with the PayPal REST API. It must never be sent to the browser.
id_token	JWT token that the server provides to the browser to initialize the SDK in an authenticated manner. Assigned to the SDK script's <code>data-sdk-client-token</code> attribute.
CMID	Client Metadata ID. Alphanumeric identifier of up to 32 characters without hyphens, generated once per checkout session. Common practice is using a UUID v4 without hyphens (32 characters), but any unique alphanumeric identifier with length ≤ 32 is valid. Acts as the correlation thread between Fraudnet, STC, the SDK, and the PayPal-Client-Metadata-Id headers.
Fraudnet	PayPal JavaScript snippet that collects device fingerprinting and browser behavior signals and transmits them to PayPal Risk. It consists of two <code><script></code> elements that must be injected only once per session.
STC	Set Transaction Context. Endpoint <code>PUT /v1/risk/transaction-contexts/{merchant_id}/{cmid}</code> through which the merchant sends buyer context to PayPal before each Create Order, enabling pre-transaction risk evaluation. It is non-blocking: errors must never stop checkout.
CBP	Chargeback Protection. PayPal commercial product that covers eligible disputes. It is a prerequisite for 3DS Risk Initiated.
PCI DSS	Payment Card Industry Data Security Standard. Security standard for handling card data. Card Fields enables SAQ A compliance because PAN/CVV never touch the merchant infrastructure.
SAQ A	Self-Assessment Questionnaire A. Simplified PCI questionnaire applicable when the merchant does not store, process, or transmit account data.
PSD2	Payment Services Directive 2. European regulation requiring Strong Customer Authentication (SCA) for certain transaction types.
SCA	Strong Customer Authentication. Strengthened buyer authentication required by PSD2; 3DS is one valid mechanism.

Term	Definition
BOPIS	Buy Online, Pickup In Store. Flow in which the buyer pays online but picks up the order in a physical store. It changes how the order shipping object is represented.
APM	Alternative Payment Method. Payment method other than credit/debit card (wallets, bank transfers, BNPL, etc.). Outside the scope of this SDD.
NFR	Non-Functional Requirement.

Part II — Solution Definition

6. Solution Overview

The solution consists of three planes with strictly bounded responsibilities and a correlation identifier (CMID) that crosses all three throughout the checkout session.

Plane	Responsibility	What It Does NOT Do
Buyer browser	Generates the CMID, injects Fraudnet, mounts the SDK, renders Card Fields, captures interactions, triggers submit, and reacts to the result.	Never knows CLIENT_SECRET, access_token, or MERCHANT_ID. Never calls the PayPal REST API directly for business operations.
Merchant backend	Protects credentials, obtains access_token through OAuth2, exposes proxy endpoints to PayPal, injects PayPal-Request-Id and PayPal-Client-Metadata-Id headers, and executes STC.	Never receives PAN or CVV. Never exposes access_token to the frontend.
PayPal REST API	Processes orders, captures, Vault tokens, Installments options, risk context (STC), and 3DS authentication.	It is the only transactional source of truth.

6.1 Solution Components

Component	Plane	Function
Card Fields	Browser	Renders card inputs as secure iframes.
Fraudnet	Browser	Device/browser telemetry.
PayPal JavaScript SDK	Browser	Orchestrates Card Fields, Installments callbacks, and 3DS execution.
OAuth2 Service	Backend	Obtains and caches access_token; exposes id_token to the frontend.
Orders Proxy	Backend	Creates and captures orders. Injects idempotency and correlation headers.
Vault Proxy	Backend	Lists and deletes tokens.
Installments Proxy	Backend	Queries financing options for saved tokens.

Component	Plane	Function
STC Proxy	Backend	Sends buyer context to /v1/risk/transaction-contexts.
PayPal REST API	PayPal	Actual processing.

6.2 Correlation Identifier: the CMID

The CMID (Client Metadata ID) is an alphanumeric identifier of up to 32 characters without hyphens, generated once per checkout session. The recommended implementation is a UUID v4 without hyphens (32 characters), but the PayPal contract accepts any unique alphanumeric identifier with length between 1 and 32 characters. It crosses the three planes:

Browser	Merchant backend	PayPal API
1. generates CMID		
2. CMID → Fraudnet "f"		
3. CMID → SDK data-client-metadata-id		
4. CMID → /api/orders body._cmid	→ PayPal-Client-Metadata-Id	→ /v2/checkout/orders
	PayPal-Client-Metadata-Id	→ /v2/checkout/orders/{id}/capture
		→ /v1/risk/transaction-contexts/{mid}/{cmid}

⚠ Without a consistent CMID across the three planes, PayPal Risk cannot correlate Fraudnet signals, STC context, and the actual transaction, which degrades the quality of risk evaluation.

7. Functional Requirements

ID	Requirement	Priority
RF-01	The system must allow the buyer to enter card data (number, expiration, CVV, name) without that data traveling to the merchant backend.	Mandatory
RF-02	The system must create an order with intent: "CAPTURE" and a mathematically consistent breakdown (item_total + tax_total + shipping - discount = amount.value).	Mandatory
RF-03	The system must execute payment capture only when the 3DS result is favorable (liabilityShift absent or equal to "POSSIBLE").	Mandatory
RF-04	The system must offer the buyer the option to save their card for future purchases (Vault-with-Purchase) through explicit consent.	Mandatory
RF-05	The system must list the saved cards of an authenticated buyer and allow payment with a token without re-entering card data.	Mandatory
RF-06	The system must allow the buyer to delete a saved card from their profile.	Mandatory
RF-07	The system must present the buyer with eligible installment financing options for their card and amount, clearly distinguishing interest-free Installments and IC2B.	Mandatory
RF-08	The system must transmit buyer context to PayPal Risk before each Create Order, without blocking checkout if an error occurs.	Mandatory
RF-09	The system must inject Fraudnet in the browser only once per checkout session, using the CMID generated at startup.	Mandatory
RF-10	The system must show the buyer a clear and actionable message when 3DS fails, without capturing the payment.	Mandatory
RF-11	The system must support the BOPIS use case by replacing the shipping object with the pickup point address.	Optional (depends on the merchant model)
RF-12	The system must allow buyer retries within the same checkout session without regenerating the CMID or reloading Fraudnet.	Mandatory

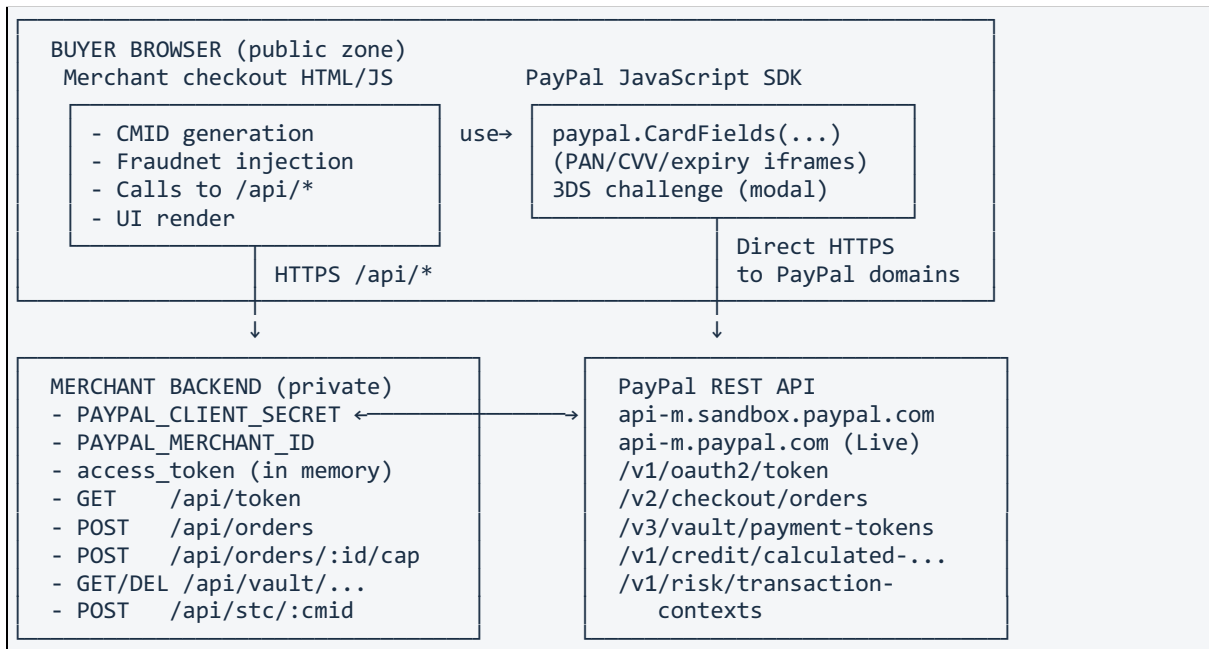
8. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Security — PCI DSS	PAN and CVV must never touch the merchant infrastructure. The integration must allow compliance under SAQ A.
NFR-02	Security — secrets	PAYPAL_CLIENT_SECRET, PAYPAL_MERCHANT_ID, and access_token must never be transmitted to the browser, stored in versioned source code, or appear in logs.
NFR-03	Security — transport	All communications (browser ↔ backend, backend ↔ PayPal) must use TLS 1.2 or higher.
NFR-04	Security — CSP	The site must declare a Content Security Policy that explicitly lists PayPal origins (www.paypal.com, www.paypalobjects.com, c.paypal.com, api-m.paypal.com).
NFR-05	Availability	Failure of auxiliary services (STC, Fraudnet) must not interrupt the payment flow. Their behavior is non-blocking.
NFR-06	Idempotency	The system must guarantee that retries of Create Order or Capture Order do not generate duplicate orders or captures by using PayPal-Request-Id.
NFR-07	Latency (p95)	The total time between submit and the user response must not exceed the merchant's internal SLOs. The access_token cache (§11.3) and non-blocking STC processing are essential to achieve this.
NFR-08	Traceability	The system must log liabilityShift, enrollment_status, authentication_status, order_id, capture_id, capture decision, and PayPal error code, without logging PAN or CVV.
NFR-09	Internationalization	The SDK must be loaded with the locale that corresponds to the target market, and error messages must be translated.
NFR-10	Accessibility	The Card Fields UI must comply with WCAG 2.1 AA: associated labels, error messages with aria-live, sufficient contrast, and no dependence on color alone to indicate invalidity.
NFR-11	Scalability	The backend must cache the access_token up to 90% of expires_in to avoid saturating /v1/oauth2/token under load.
NFR-12	Browser compatibility	The integration must validate cardField.isEligible() before rendering and degrade gracefully when the browser does not support Card Fields.

ID	Category	Requirement
NFR-13	Auditability	Each transaction must be reconstructable from logs: CMID, PayPal-Request-Id, order_id, invoice_id, custom_id.

9. Solution Architecture

9.1 Logical Diagram



9.2 Non-Negotiable Architecture Rules

1. `PAYPAL_CLIENT_SECRET` resides only in backend environment variables. It is never versioned and never sent to the browser.
2. The browser never calls `api-m.paypal.com` directly for business operations. All REST API communication goes through the merchant backend.
3. The browser never receives the `access_token`. It receives only `clientId` (public) and `id_token`.
4. PAN and CVV never touch the merchant backend. Card Fields isolates them inside `<iframe>` elements from the PayPal domain.
5. Cart, buyer, and shipping address data are never hardcoded in frontend code.

9.3 Security Boundary by Asset

Asset	Frontend	Backend	PayPal API
CLIENT_ID	Yes	Yes	—
CLIENT_SECRET	No	Yes (env)	—

Asset	Frontend	Backend	PayPal API
MERCHANT_ID	No	Yes (env)	—
access_token	No	Yes (memory)	—
id_token	Yes (to SDK)	Yes	—
PAN / CVV	PayPal iframe	No	Yes
PAYMENT_METHOD_TOKEN	Yes (reference)	Yes	Yes
CMID	Yes (generated)	Yes (received in body)	Yes (header)

9.4 Architectural Pattern: Backend-for-Frontend (BFF) over PayPal

The merchant backend implements the Backend-for-Frontend pattern over the PayPal REST API. The `/api/*` endpoints are not a generic API; they exist for the specific needs of checkout and protect secrets, idempotency, and correlation headers that the frontend must not handle.

10. Prerequisites and Environment Configuration

10.1 Required Commercial Enablements

Capability	Applies To
ACDC with Card Fields	Card processing through the SDK.
Vault v3	Payment method tokenization.
Installments (Buyer / Seller financed)	Installment payments (interest-free and IC2B) in the target market.
Chargeback Protection (CBP)	Activation of 3DS Risk Initiated.
Fraudnet + Set Transaction Context	Access to the /v1/risk/transaction-contexts endpoint.

10.2 Backend Credentials and Configuration

Environment Variable	Source	Visibility
PAYPAL_CLIENT_ID	PayPal Developer Dashboard → merchant app.	Public (may go to the frontend).
PAYPAL_CLIENT_SECRET	PayPal Developer Dashboard → merchant app.	Secret — backend only.
PAYPAL_MERCHANT_ID	PayPal merchant account profile.	Private — backend only (required for STC).
PAYPAL_API_BASE	https://api-m.sandbox.paypal.com (Sandbox) · https://api-m.paypal.com (Live).	Private.

 Sandbox and Live credentials are different. PAYPAL_API_BASE, PAYPAL_CLIENT_ID, PAYPAL_CLIENT_SECRET, and PAYPAL_MERCHANT_ID must belong to the same environment. Mixing them produces 401 Unauthorized on the first OAuth call.

10.3 Environment Requirements

- TLS / HTTPS is mandatory in production. Card Fields and Fraudnet do not operate over HTTP.
- Content Security Policy (CSP): must allow at minimum script-src <https://www.paypal.com> <https://www.paypalobjects.com> <https://c.paypal.com>;

frame-src https://www.paypal.com; connect-src https://api-m.paypal.com
https://api-m.sandbox.paypal.com

- Web Crypto API support in the browser (for `crypto.randomUUID()`). To support legacy browsers, implement a fallback (see §12).

Part III — Detailed Design

11. OAuth2 Authentication and id_token Issuance

The merchant backend is the only component authorized to perform OAuth2 communication with PayPal. The purpose of this step is to obtain both: an access_token for the backend to call the PayPal REST API, and an id_token that the backend forwards to the browser to initialize the SDK.

11.1 Request

```
POST {PAYPAL_API_BASE}/v1/oauth2/token
Authorization: Basic <BASE64(CLIENT_ID:CLIENT_SECRET)>
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials&response_type=id_token
```

 response_type=id_token must be present. Without it, the response does not include the id_token required by the frontend SDK.

11.2 Response (Relevant Fields)

```
{
  "access_token": "<ACCESS_TOKEN>",
  "id_token": "<ID_TOKEN>",
  "expires_in": 32400,
  "token_type": "Bearer"
}
```

11.3 access_token Cache

Practice	Rationale
Cache the access_token in backend process memory until 90% of expires_in and refresh it proactively.	Avoids unnecessary calls to /v1/oauth2/token and reduces latency on the Create Order critical path.
Do not persist the access_token to disk, database, or logs.	Bearer token with transactional power; exposure compromises the account.
Refresh immediately upon a 401 from any call and retry once.	Covers the edge case of a token revoked by credential rotation.

11.4 Endpoint Exposed to the Frontend

```
GET /api/token  
→ 200 OK  
{  
  "clientId": "<PAYPAL_CLIENT_ID>",  
  "idToken": "<ID_TOKEN>"  
}
```

⚠ This endpoint must never return the `access_token`. If it is accidentally delivered to the browser, any visitor can operate the merchant's PayPal account.

11.5 Reference Implementation (Node.js)

```
async function generateAccessToken() {  
  const credentials = Buffer.from(  
    `${process.env.PAYPAL_CLIENT_ID}:${process.env.PAYPAL_CLIENT_SECRET}`  
  ).toString("base64");  
  const response = await fetch(`${process.env.PAYPAL_API_BASE}/v1/oauth2/token`, {  
    method: "POST",  
    headers: {  
      "Authorization": `Basic ${credentials}`,  
      "Content-Type": "application/x-www-form-urlencoded" },  
    body: "grant_type=client_credentials&response_type=id_token"  
  });  
  if (!response.ok) throw new Error(`OAuth failed: ${response.status}`);  
  return response.json(); // { access_token, id_token, expires_in, ... }  
}
```

12. Client Metadata ID (CMID) Generation

The CMID is the first data element materialized when the browser initializes checkout. It is a unique alphanumeric identifier of up to 32 characters without hyphens that acts as the correlation thread across four components:

1. The "f" field in the Fraudnet configuration script.
2. The data-client-metadata-id attribute in the PayPal SDK script.
3. The STC endpoint URL (/v1/risk/transaction-contexts/{merchant_id}/{cmid}).
4. The PayPal-Client-Metadata-Id header sent to Create Order and Capture Order.

12.1 Lifecycle Rules

Rule	Detail
Unique per checkout session	Generated once when the checkout page initializes.
Persistent across retries	If the buyer fails a payment and retries within the same session, the CMID is not regenerated. Fraudnet is not reloaded either.
Persistent across payment methods	If the buyer switches between new card and saved card in the same session, the CMID is retained.
Regenerated for a new transaction	Only after the prior transaction is completed or definitively cancelled is a new CMID generated and Fraudnet reloaded.
Propagated server-side by the backend	The frontend sends it to the backend in the Create Order/Capture Order body; the backend forwards it to PayPal as an HTTP header.

12.2 Reference Implementation

The recommended implementation is a UUID v4 without hyphens, which uses all 32 available characters and provides the highest possible entropy within the limit. Other strategies (timestamp + random suffix, truncated ULID, deterministic session hash) are technically valid as long as the result has between 1 and 32 characters, contains only alphanumeric characters, and is unique per checkout session.

```
function generateCMID() {
  if (typeof crypto !== "undefined" && typeof crypto.randomUUID === "function") {
    return crypto.randomUUID().replace(/-/g, ""); // 32 hex characters
  }
  // Fallback for environments without Web Crypto API
  return "xxxxxxxx4xxxymxxxxxxxxxxxxx".replace(/[xy]/g, (c) => {
    const r = (Math.random() * 16) | 0;
    return (c === "x" ? r : (r & 0x3) | 0x8).toString(16);
  });
}
```

```
}  
const cmid = generateCMID(); // e.g. "a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4"
```

💡 Although the PayPal contract accepts up to 32 characters, using maximum-length identifiers (UUID v4 without hyphens) is recommended: it maximizes uniqueness and removes ambiguity around the range accepted by the API.

12.3 Validation

```
const isValidCMID = (cmid) =>  
  typeof cmid === "string" && cmid.length >= 1 && cmid.length <= 32 &&  
  /^[0-9a-zA-Z]+$/.test(cmid);
```

13. Fraudnet Integration

Fraudnet is the first risk service initialized in the browser, immediately after generating the CMID and before any backend or SDK call. It collects device fingerprinting, browser behavior signals, and session telemetry that PayPal Risk uses to evaluate the transaction.

13.1 Composition

Fraudnet is implemented with two `<script>` elements injected into the DOM:

13.1.1 Configuration Script (JSON)

```
<script type="application/json"
  fncls="fnparams-dede7cc5-15fd-4c75-a9f4-36c430ee3a99">
{
  "f": "<CMID>",
  "s": "<MERCHANT_SHORT_NAME>_<MERCHANT_ID>_ACDC"
}
</script>
```

Attribute / field	Value	Description
type	application/json	Constant — do not change.
fncls	fnparams-dede7cc5-15fd-4c75-a9f4-36c430ee3a99	Constant — Fraudnet configuration identifier.
"f"	The CMID generated in §12	Alphanumeric identifier of up to 32 characters without hyphens.
"s"	<MERCHANT_SHORT_NAME>_<MERCHANT_ID>_ACDC	Merchant and product identifier. Fixed format. For Card Fields, the product is always ACDC.

13.1.2 Library Script

```
<script type="text/javascript" src="https://c.paypal.com/da/r/fb.js"></script>
```

 Order matters. The JSON configuration script must be present in the DOM before fb.js executes. Inject the configuration first, then the library.

13.2 Dynamic Implementation

```
let fraudnetLoaded = false;
function loadFraudnet({ cmid, merchantShortName, merchantId }) {
  if (fraudnetLoaded) return;
  fraudnetLoaded = true;
  const config = document.createElement("script");
  config.type = "application/json";
  config.setAttribute("fncls", "fnparams-dede7cc5-15fd-4c75-a9f4-36c430ee3a99");
  config.textContent = JSON.stringify({ f: cmid, s:
`${merchantShortName}_${merchantId}_ACDC` });
  document.body.appendChild(config);
  const lib = document.createElement("script");
  lib.type = "text/javascript";
  lib.src = "https://c.paypal.com/da/r/fb.js";
  document.body.appendChild(lib);
}
```

13.3 Operational Rules

Rule	Detail
One load per checkout session.	Repeating the injection invalidates the Fraudnet session and produces inconsistent signals.
Covers new card and saved card.	The same Fraudnet session applies to both flows; do not reload when changing payment method.
Do not reload after a failed attempt.	A retry within the same session must use the same Fraudnet load and the same CMID.
Reload only for a new transaction.	After a completed or definitively cancelled transaction, generate a new CMID and invoke loadFraudnet again.
CSP-friendly.	The https://c.paypal.com origin must be included in the site's Content Security Policy script-src.

14. Set Transaction Context (STC)

STC allows the merchant to send buyer context to PayPal Risk before each Create Order. PayPal correlates it with the Fraudnet session (through the CMID) for pre-transaction risk evaluation. The operation is non-blocking: any error must be logged but must never stop the checkout flow.

14.1 Endpoint

```
PUT {PAYPAL_API_BASE}/v1/risk/transaction-contexts/<MERCHANT_ID>/<CMID>
Authorization: Bearer <ACCESS_TOKEN>
Content-Type: application/json
```

14.2 Body — Generic Retail Set

```
{
  "additional_data": [
    { "key": "sender_account_id", "value": "<BUYER_ID_IN_MERCHANT_PLATFORM>" },
    { "key": "sender_first_name", "value": "<BUYER_FIRST_NAME>" },
    { "key": "sender_last_name", "value": "<BUYER_LAST_NAME>" },
    { "key": "sender_email", "value": "<BUYER_EMAIL>" },
    { "key": "sender_phone", "value": "<DIGITS_ONLY_PHONE>" },
    { "key": "sender_country_code", "value": "<ISO_ALPHA2_COUNTRY>" },
    { "key": "sender_create_date", "value": "<USER_SIGNUP_DATE>" },
    { "key": "highrisk_txn_flag", "value": 0 },
    { "key": "vertical", "value": "<BUSINESS_VERTICAL>" },
    { "key": "cd_string_one", "value": "<USER_TRUST_LEVEL>" }
  ]
}
```

14.3 Field Reference

Field	Type	Description	Accepted Values
sender_account_id	string	Unique buyer identifier in the merchant platform.	Stable alphanumeric value across sessions.
sender_first_name	string	Buyer's registered first name.	Alphanumeric.
sender_last_name	string	Buyer's registered last name.	Alphanumeric.
sender_email	string	Buyer's validated email.	RFC 5322 format.
sender_phone	string	Buyer's phone number, digits only, no formatting.	[0-9]+

Field	Type	Description	Accepted Values
sender_country_code	string	Buyer's country in ISO 3166-1 Alpha-2.	MX, US, BR, etc.
sender_create_date	string	User creation date in the merchant platform.	Accepted formats: yyyy-mm-ddThh:mm:ss.000-00:00 yyyy-mm-ddThh:mm:ssZ yyyy-mm-dd yyyymmdd
highrisk_txn_flag	number	Indicates whether the transaction is high risk.	0 = normal, 1 = high risk. Not a string.
vertical	string	Business vertical.	Retail, Travel, Gaming, etc. (check the industry pack with the Integration Engineer).
cd_string_one	string	Buyer trust level assigned by the platform.	"1" trusted / "0" unknown / "2" untrusted.

💡 Industry packs: The set above is the generic Retail set. Verticals such as Travel, OTAs, Financial Services, Gaming, and regulated platforms require additional specific fields. Request the corresponding industry pack from the assigned Integration Engineer.

14.4 Response Handling — Non-Blocking Behavior

HTTP Status	Meaning	Action
200	OK. Context registered.	Continue with Create Order.
400	Invalid body (incorrect field type or format).	Log the error with detail, continue with Create Order.
401	Missing permissions or expired access_token.	Refresh token, log, continue with Create Order.
5xx	Internal PayPal error.	Log, continue with Create Order.

🔒 STC can never block checkout. An STC failure reduces the quality of risk evaluation but does not prevent transaction processing.

14.5 Reference Implementation

14.5.1 Backend (Node.js)

```
app.post("/api/stc/:cmid", async (req, res) => {
  try {
    const { cmid } = req.params;
    const { access_token } = await generateAccessToken();
    const response = await fetch(
      `${process.env.PAYPAL_API_BASE}/v1/risk/transaction-
      contexts/${process.env.PAYPAL_MERCHANT_ID}/${cmid}`,
      { method: "PUT",
        headers: { "Authorization": `Bearer ${access_token}`, "Content-Type":
        "application/json" },
        body: JSON.stringify({ additional_data: buildAdditionalDataFromSession(req) }) }
    );
    res.status(200).json({ success: response.ok, status: response.status });
  } catch (err) {
    console.warn("STC failed (non-blocking):", err);
    res.status(200).json({ success: false }); // Do not block checkout
  }
});
```

14.5.2 Frontend

```
async function callSTC(cmId) {
  try { await fetch(`/api/stc/${cmId}`, { method: "POST" }); }
  catch (err) { console.warn("STC could not complete (non-blocking):", err); }
}
```

 STC is called before each Create Order, regardless of payment method (new card or saved card).

15. Loading the PayPal JavaScript SDK

With the CMID generated, Fraudnet injected, and the id_token obtained from the backend, the next step is to load the PayPal SDK in the browser. SDK loading must happen after §12, §13, and retrieval of clientId + idToken (§11.4).

15.1 Building the SDK URL

```
const url = new URL("https://www.paypal.com/sdk/js");
url.searchParams.set("client-id", clientId);
url.searchParams.set("currency", "<ISO_4217_CURRENCY>");
url.searchParams.set("locale", "<BCP47_LOCALE>");
url.searchParams.set("components", "card-fields"); // card-fields only

const script = document.createElement("script");
script.src = url.toString();
script.setAttribute("data-sdk-client-token", idToken);
script.setAttribute("data-client-metadata-id", cmid); // same CMID as Fraudnet and STC
script.onload = () => { renderCardFields(); };
document.body.appendChild(script);
```

Parameter	Value	Notes
client-id	<PAYPAL_CLIENT_ID>	Received from the backend in GET /api/token.
currency	MXN, USD, BRL, etc.	Main checkout currency in ISO 4217.
locale	es_MX, en_US, pt_BR, etc.	SDK UI language. BCP-47 format with underscore.
components	card-fields	Component to load. For this solution, only card-fields.

 The data-client-metadata-id must be exactly the same value used in Fraudnet's "f" field (§13), the STC URL (§14), and the PayPal-Client-Metadata-Id header. Any divergence breaks risk correlation.

16. Card Fields HTML Structure and Styles

16.1 DOM Containers

```
<div id="card-number-field-container"></div>
<div id="card-expiry-field-container"></div>
<div id="card-cvv-field-container"></div>
<div id="card-name-field-container"></div>
<select id="installments-select"><option value="">No installments</option></select>
<label><input type="checkbox" id="vault" /> Save card for future purchases</label>
<button id="card-field-submit" type="button">Pay</button>
<div id="payment-result" role="status" aria-live="polite"></div>
```

16.2 Rendering

```
function renderCardFields() {
  const cardField = paypal.CardFields({
    style: styleObject, createOrder: createOrderFn,
    onApprove: onApproveFn, onError: onErrorFn, installments: installmentsBlock
  });
  if (!cardField.isEligible()) return;
  cardField.NumberField().render("#card-number-field-container");
  cardField.ExpiryField().render("#card-expiry-field-container");
  cardField.CVVField().render("#card-cvv-field-container");
  cardField.NameField().render("#card-name-field-container");
}
```

16.3 Layer 1 — Container CSS

The `<div>` hosting each `iframe` lives in the merchant domain. Its CSS controls dimensions, border, and layout. PayPal publishes a reference stylesheet:

```
<link rel="stylesheet"
href="https://www.paypalobjects.com/webstatic/en_US/developer/docs/css/cardfields.css">
```

16.4 Layer 2 — SDK styleObject

```
const styleObject = {
  input: { "font-size": "14px", "font-family": "system-ui, sans-serif", "color":
"<NORMAL_TEXT_COLOR>" },
  ".invalid": { "color": "<INVALID_TEXT_COLOR>" },
  ":focus": { "color": "<FOCUS_TEXT_COLOR>" },
  ":hover": { "color": "<HOVER_TEXT_COLOR>" }
};
```

Selector	State
input	Base field state.
.invalid	The value fails SDK internal validation.
:focus	The field has focus.
:hover	The cursor is over the field.

CSS properties allowed inside the iframe:

color, font, font-family, font-size, font-size-adjust, font-stretch, font-style, font-variant, font-weight, line-height, letter-spacing, opacity, outline, text-shadow, transition, padding, padding-top, padding-right, padding-bottom, padding-left.

💡 Accessibility (NFR-10): Do not use color as the only invalidity indicator. Always accompany it with text and/or an icon for users with low vision or color blindness. Associate error messages through `aria-live="polite"`.

17. Order Creation and Capture

17.1 Mandatory HTTP Headers

Header	Create	Capture	Description
Authorization: Bearer <ACCESS_TOKEN>	Yes	Yes	Bearer token obtained in §11.
Content-Type: application/json	Yes	Yes	Body is JSON.
PayPal-Request-Id: <UUID>	Yes	Yes	Idempotency. The same UUID must be used for Create Order and its associated Capture. Generate a new one per new transaction.
PayPal-Client-Metadata-Id: <CMID>	Yes	Yes	Links the transaction with Fraudnet and STC. Same CMID as Fraudnet's "f" and the STC URL.

17.2 Create Order — Complete Payload (New Card with Vault)

```
{
  "intent": "CAPTURE",
  "application_context": {
    "brand_name": "<MERCHANT_VISIBLE_NAME>", "locale": "<BCP47_LOCALE>",
    "shipping_preference": "SET_PROVIDED_ADDRESS", "user_action": "PAY_NOW",
    "return_url": "<MERCHANT_REAL_RETURN_URL>",
    "cancel_url": "<MERCHANT_REAL_CANCEL_URL>"
  },
  "payer": {
    "email_address": "<BUYER_EMAIL>",
    "name": { "given_name": "<BUYER_FIRST_NAME>", "surname": "<BUYER_LAST_NAME>" },
    "phone": { "phone_type": "MOBILE", "phone_number": { "national_number":
"<DIGITS_ONLY_PHONE>" } }
  },
  "purchase_units": [{
    "invoice_id": "<MERCHANT_UNIQUE_INVOICE_ID>", "custom_id":
"<MERCHANT_INTERNAL_ORDER_ID>",
    "description": "<SHORT_ORDER_DESCRIPTION>",
    "amount": {
      "currency_code": "<ISO_4217_CURRENCY>", "value": "<CART_TOTAL>",
      "breakdown": {
        "item_total": { "currency_code": "<CURRENCY>", "value": "<SUM_UNIT_AMOUNT_X_QTY>"
      },
      "tax_total": { "currency_code": "<CURRENCY>", "value": "<SUM_TAX_X_QTY>" },
      "shipping": { "currency_code": "<CURRENCY>", "value": "<SHIPPING_COST>" },
      "discount": { "currency_code": "<CURRENCY>", "value": "<APPLIED_DISCOUNT>" }
    }
  }],
  "items": [{
```

```

    "name": "<PRODUCT_NAME>", "description": "<PRODUCT_DESCRIPTION>", "sku":
"<CATALOG_SKU>",
    "quantity": "<QUANTITY>",
    "unit_amount": { "currency_code": "<CURRENCY>", "value": "<UNIT_PRICE_WITHOUT_TAX>"
},
    "tax": { "currency_code": "<CURRENCY>", "value": "<TAX_PER_UNIT>" },
    "category": "PHYSICAL_GOODS"
  }],
  "shipping": {
    "name": { "full_name": "<RECIPIENT_FULL_NAME>" },
    "address": {
      "address_line_1": "<STREET_AND_NUMBER>", "address_line_2":
"<NEIGHBORHOOD_OR_REFERENCE>",
      "admin_area_2": "<CITY_OR_MUNICIPALITY>", "admin_area_1": "<STATE_CODE>",
      "postal_code": "<POSTAL_CODE>", "country_code": "<ISO_ALPHA2_COUNTRY>"
    }
  }
},
"payment_source": { "card": { "attributes": {
  "customer": { "id": "<MERCHANT_AUTHENTICATED_USER_ID>" },
  "vault": {
    "store_in_vault": "ON_SUCCESS", "usage_type": "MERCHANT",
    "customer_type": "CONSUMER", "permit_multiple_payment_tokens": true
  }
}
}}}
}

```

17.3 Rationale for Each Block

Block	Why It Is Mandatory in Production
application_context.shipping_preference: SET_PROVIDED_ADDRESS	Tells PayPal to use the address included in purchase_units[].shipping. Improves the quality of risk signals.
application_context.return_url / cancel_url	Required for 3DS flow redirects when requested by the issuing bank. Must be real URLs in the merchant domain.
payer.email_address, payer.name, payer.phone	Identify the buyer for risk evaluation and dispute support.
invoice_id	Unique merchant identifier for accounting reconciliation and logical idempotency. Prevents duplicate orders during retries.
custom_id	Additional merchant internal ID (for example, back-office order ID).
breakdown + items	The total amount must be mathematically consistent with the breakdown and line items. This is required for Installments.
items[].tax	Line-level VAT/tax, required for tax_total to reconcile.

Block	Why It Is Mandatory in Production
items[].category	PHYSICAL_GOODS, DIGITAL_GOODS, or DONATION. Influences risk processing.
shipping.address	Required when shipping_preference is SET_PROVIDED_ADDRESS.
payment_source.card.attributes.customer.id	Links the card to the buyer in Vault. Enables secure reuse.
payment_source.card.attributes.vault	Enables saving in Vault after successful payment completion. If Vault-with-Purchase is not desired, omit this block.

17.4 Breakdown Validation Rules

```
amount.value === item_total + tax_total + shipping - discount
item_total    === Σ (item.unit_amount × item.quantity) per line
tax_total     === Σ (item.tax × item.quantity) per line
```

If the values do not reconcile, PayPal responds with 422 UNPROCESSABLE_ENTITY and details the inconsistent field.

17.5 Create Order with Saved Card (Token)

The base payload is identical to §17.2 (same payer, purchase_units, breakdown, items, shipping, application_context). Only payment_source changes:

```
{
  "payment_source": {
    "token": {
      "id": "<VAULT_PAYMENT_METHOD_TOKEN>",
      "type": "PAYMENT_METHOD_TOKEN"
    }
  }
}
```

Returned status	Action
COMPLETED	The order was auto-captured. Do not call /capture.
APPROVED	Manual capture required. Call POST /v2/checkout/orders/{id}/capture.

17.6 Propagating the CMID to the Backend Header

```

// Frontend – inside the SDK createOrder callback
async function createOrderFn() {
  await callSTC(cmid); // §14
  const body = buildOrderPayload(); // §17.2
  body._cmid = cmid; // Convention: internal control field
  const response = await fetch("/api/orders", {
    method: "POST", headers: { "Content-Type": "application/json" },
    body: JSON.stringify(body)
  });
  return (await response.json()).id;
}

// Backend – POST /api/orders route
app.post("/api/orders", async (req, res) => {
  const { _cmid, ...orderPayload } = req.body;
  const { access_token } = await generateAccessToken();
  const requestId = req.headers["x-request-id"] || generateRequestId();
  const response = await fetch(`${process.env.PAYPAL_API_BASE}/v2/checkout/orders`, {
    method: "POST",
    headers: { "Authorization": `Bearer ${access_token}`, "Content-Type":
"application/json",
      "PayPal-Request-Id": requestId, "PayPal-Client-Metadata-Id": _cmid },
    body: JSON.stringify(orderPayload)
  });
  res.status(response.status).json(await response.json());
});

```

18. Installments — MSI and IC2B

 Correct classification: Do not infer Installments/IC2B from the number of months. Always check `total_consumer_fee.value`.

Mode	Who Pays the Fee	<code>total_consumer_fee.value</code>	<code>fee_reference_id</code>	Suggested UI Label
Interest-free Installments	Merchant (absorbs the cost)	"0.00"	Usually optional	"X interest-free months"
IC2B (Installments Cost To Buyer)	Buyer	> "0.00"	Mandatory	"X months — fee \$Y <CURRENCY>"

18.1 New Card — SDK Callbacks

```
const cardField = paypal.CardFields({
  installments: {
    onInstallmentsRequested: () => ({
      financingCountryCode: "<ISO_ALPHA2_COUNTRY>",
      amount: "<CART_TOTAL>",
      currencyCode: "<ISO_4217_CURRENCY>",
      billingCountryCode: "<ISO_ALPHA2_BILLING_COUNTRY>",
      includeBuyerInstallments: true
    }),
    onInstallmentsAvailable: ({ financing_options }) => {
      const options = (financing_options || [])
        .filter((o) => o.product === "CARD_ISSUER_INSTALLMENTS")
        .flatMap((o) => o.qualifying_financing_options);
      options.forEach((opt) => {
        const term = opt.credit_financing.term;
        const isMSI = parseFloat(opt.total_consumer_fee.value) === 0;
        const label = term === 1 ? "Pay in full"
          : isMSI ? `${term} interest-free months`
          : `${term} months — fee ${opt.total_consumer_fee.value}
            ${opt.total_consumer_fee.currency_code}`;
        // Store: { term, intervalDuration: opt.credit_financing.interval_duration,
        //         feeReferenceId: opt.fee_reference_id }
      });
    },
    onInstallmentsError: (err) => { /* log + fallback UI */ }
  }
});
```

18.2 Submit with Installments

```
async function submitWithSelectedInstallments(cardField) {
  const selected = readSelectedOptionFromUI();
  if (selected.term === 1) { await cardField.submit({}); return; }
  await cardField.submit({
    installments: {
      term: selected.term,
      intervalDuration: selected.intervalDuration, // "P1M" = monthly (ISO 8601)
      feeReferenceId: selected.feeReferenceId // MANDATORY in IC2B
    }
  });
}
```

 If the term is 1 (pay in full), do not send the installments block. Send an empty object {} to submit.

18.3 Saved Card — calculated-financing-options

```
{
  "financing_country_code": "<ISO_ALPHA2_COUNTRY>",
  "transaction_amount": { "value": "<CART_TOTAL>", "currency_code": "<ISO_4217_CURRENCY>"
},
  "funding_instrument": {
    "type": "TOKEN",
    "token": { "type": "PAYMENT_METHOD_TOKEN", "payment_method_token":
"<VAULT_PAYMENT_METHOD_TOKEN>" }
  },
  "flow_context": { "attributes": ["FEE_POLICY_CHARGE_CONSUMER"] }
}
```

 The FEE_POLICY_CHARGE_CONSUMER attribute in flow_context.attributes is required for the response to include IC2B options. Without it, the response may contain only interest-free Installments.

18.4 Create Order with Token + Installments (REST uses snake_case)

```
{
  "payment_source": {
    "token": {
      "id": "<PAYMENT_METHOD_TOKEN>", "type": "PAYMENT_METHOD_TOKEN",
      "attributes": {
        "installments": {
          "term": "<MONTH_TERM>",
          "interval_duration": "<ISO_8601_INTERVAL_DURATION>",
          "fee_reference_id": "<OPTION_FEE_REFERENCE_ID>"
        }
      }
    }
  }
}
```

```
}  
}
```

Context	Format	Example
SDK callback (<code>cardField.submit</code>)	camelCase	intervalDuration, feeReferenceId
REST API (<code>payment_source.token.attributes.installments</code>)	snake_case	interval_duration, fee_reference_id

 Mixing formats is the most frequent cause of UNPROCESSABLE_ENTITY errors in Installments.

19. Vault — Card Tokenization

Vault securely stores the payment method and returns an opaque `PAYMENT_METHOD_TOKEN` that the merchant reuses to charge future transactions without requiring the buyer to re-enter their data.

19.1 customer.id — Stable Buyer Identifier

- Stable: the same identifier across the user's sessions.
- Deterministic: generable from the user record (not random per session).
- Unique per buyer: two distinct buyers never share a customer.id.

 Recommendation: use the user's internal identifier in the merchant database or a deterministic hash of it. Avoid emails (they change over time) or temporary test identifiers.

19.2 Vault-with-Purchase

Attribute	Value	Meaning
<code>store_in_vault</code>	<code>ON_SUCCESS</code>	Saves the card only if the transaction completes successfully. Failed cards are not saved.
<code>usage_type</code>	<code>MERCHANT</code>	Future charges initiated by the merchant (vs <code>CUSTOMER</code> , initiated by the buyer).
<code>customer_type</code>	<code>CONSUMER</code>	Individual person (vs <code>BUSINESS</code>).
<code>permit_multiple_payment_tokens</code>	<code>true</code>	Allows multiple tokens for the same customer.id (multiple cards).

19.3 Listing, Charging, and Deleting Tokens

```
// List
GET {PAYPAL_API_BASE}/v3/vault/payment-tokens?customer_id=<CUSTOMER_ID>

// Response fields useful for the UI:
//   brand, last_digits, expiry → display "VISA ....1234 (exp. 03/27)"

// Delete
DELETE {PAYPAL_API_BASE}/v3/vault/payment-tokens/<PAYMENT_METHOD_TOKEN>
```

19.4 Test Cards for Vault (Sandbox)

Brand	Number	Notes
Visa (Bancomer)	4772129056533503	Credit card issued by Bancomer (Mexico).
Mastercard	5288775404117508	Credit card.
Amex	376680816376961	4-digit CVV.

20. 3DS Risk Initiated and liabilityShift

3-D Secure (3DS) is a protocol where the buyer's issuing bank verifies their identity. When 3DS succeeds, liability for fraudulent chargebacks shifts from the merchant to the issuing bank.

20.1 Modes

Mode	Who Decides When to Trigger 3DS	Required Product	Coverage in This SDD
Risk Initiated	PayPal risk engine	Chargeback Protection (CBP)	Covered. The merchant does not send 3DS instructions; it validates the result in onApprove.
Merchant Initiated	Merchant	Fraud Protection (FP)	Not covered. The merchant configures payment_source.card.attributes.verification in Create Order.

20.2 Mandatory Logic in onApprove

```

async function onApprove(data) {
  if (!data.liabilityShift) {
    // No 3DS challenge – the risk engine did not consider it necessary.
    await capture(data.orderID);
    return;
  }
  if (data.liabilityShift === "POSSIBLE") {
    // Successful 3DS – the issuing bank assumes chargeback liability.
    await capture(data.orderID);
    return;
  }
  // liabilityShift present with any other value ("N", "U", etc.) → do not capture.
  showError("Authentication could not be completed. Please try another payment method.");
}

```

20.3 Decision Table by liabilityShift Value

Value	Meaning	Action
undefined (absent)	There was no 3DS challenge.	Capture.
"POSSIBLE"	Successful 3DS. Liability shifted to issuer.	Capture.

Value	Meaning	Action
"N"	Authentication failed.	Do not capture. Show error.
"U" / others	Unavailable or rejected.	Do not capture. Show error.

 Do not force 3DS from the backend under the Risk Initiated model. Mixing Risk Initiated with Merchant Initiated instructions produces undefined behavior.

Part IV — Integration and Operations

21. End-to-End Orchestration

21.1 Moment 1 — Checkout Initialization (Once per Session)

```

1. cmid = generateCMID() [§12]
2. loadFraudnet({ cmid, merchantShortName, merchantId }) [§13]
3. { clientId, idToken } = GET /api/token [§11]
4. (Optional) List saved tokens:
   GET /api/vault/payment-tokens?customer_id=<...> [§19.3]
5. Load SDK with data-client-metadata-id = cmid [§15]
6. Render Card Fields [§16, §18, §20]

```

21.2 Moment 2 — Before Each Create Order (New or Saved Card)

```

1. await callSTC(cmid) [§14]
2. Build Create Order payload [§17.2 or §17.5]
3. POST /api/orders + body._cmid = cmid [§17.6]
   → backend injects:
     PayPal-Request-Id: <TRANSACTION_UUID>
     PayPal-Client-Metadata-Id: <CMID>

```

21.3 Moment 3 — Capture Decision (in onApprove)

```

if (!data.liabilityShift) → capture [§20]
else if (data.liabilityShift === "POSSIBLE") → capture
else → do not capture

```

💡 Saved card with auto-capture: After Create Order with payment_source.token, call GET /api/orders/:id and validate status (§17.5). Do not invoke /capture if it is already COMPLETED.

21.4 Moment 4 — Capture

```

POST /api/orders/<ORDER_ID>/capture + body._cmid = cmid
→ backend injects:
  PayPal-Request-Id: <SAME_UUID_AS_CREATE_ORDER>
  PayPal-Client-Metadata-Id: <CMID>

```

21.5 Mnemonic Rule

Component	Frequency
CMID + Fraudnet	Once per checkout session.
STC + PayPal-Client-Metadata-Id	On every payment attempt (new card or saved card).
PayPal-Request-Id	Same UUID for Create Order and its associated Capture; new one per transaction.

22. Specialized Use Case — BOPIS

BOPIS: the buyer pays online but picks up the order at a physical store. Payment is processed identically to a standard ACDC flow, but the shipping object inside `purchase_units` is replaced with the pickup point data, not the buyer's address.

22.1 full_name Convention

The `shipping.name.full_name` must carry the prefix S2S (Ship To Store, uppercase, with a space after it) followed by the store name. This convention allows PayPal Risk to identify the flow as in-store pickup.

Field	Delivery (home delivery)	BOPIS (store pickup)
shipping.name.full_name	<BUYER_NAME>	S2S <STORE_NAME>
shipping.address	Buyer's address	Physical store address

```
"shipping": {
  "name": { "full_name": "S2S <STORE_NAME>" },
  "address": {
    "address_line_1": "<STORE_STREET_AND_NUMBER>",
    "address_line_2": "<STORE_NEIGHBORHOOD_OR_REFERENCE>",
    "admin_area_2": "<STORE_CITY_OR_MUNICIPALITY>",
    "admin_area_1": "<STATE_CODE>",
    "postal_code": "<STORE_POSTAL_CODE>",
    "country_code": "<ISO_ALPHA2_COUNTRY>"
  }
}
```

💡 For specific implementation details (store catalog, pickup-hour validations, WMS/POS integration), contact the Integration Engineer.

23. Integration Points (Endpoint Map)

Operation	Merchant backend (suggested)	PayPal API
OAuth token for SDK	GET /api/token	POST /v1/oauth2/token
Create order	POST /api/orders	POST /v2/checkout/orders
Capture order	POST /api/orders/:id/capture	POST /v2/checkout/orders/{id}/capture
Query order	GET /api/orders/:id	GET /v2/checkout/orders/{id}
List saved tokens	GET /api/vault/payment-tokens?customer_id=...	GET /v3/vault/payment-tokens
Delete token	DELETE /api/vault/payment-tokens/:id	DELETE /v3/vault/payment-tokens/{id}
Financing options (token)	POST /api/credit/financing-options	POST /v1/credit/calculated-financing-options
Set Transaction Context	POST /api/stc/:cmid	PUT /v1/risk/transaction-contexts/{merchant_id}/{cmid}

24. Security Considerations

24.1 Credential Custody (NFR-02)

Asset	Storage	Transmission
PAYPAL_CLIENT_SECRET	Backend environment variable, managed by a secrets manager. Never in source code.	Backend only → /v1/oauth2/token encoded in Authorization: Basic.
PAYPAL_MERCHANT_ID	Backend environment variable.	Backend only → STC URL and Fraudnet configuration.
access_token	Backend process memory. Cache with TTL lower than expires_in. Never persisted.	Backend only → PayPal REST API in Authorization: Bearer.
id_token	Generated dynamically, transmitted to the frontend in response to GET /api/token.	Backend → frontend (response body) → SDK (data-sdk-client-token attribute).

24.2 PCI Isolation / CSP / Logging

- PCI Isolation (NFR-01): Card Fields renders inputs as iframes from the paypal.com domain. PAN and CVV never enter the merchant DOM. The merchant can certify under SAQ A.
- Transport (NFR-03): TLS 1.2 minimum in production.
- Minimum recommended CSP: script-src 'self' https://www.paypal.com https://www.paypalobjects.com https://c.paypal.com; frame-src https://www.paypal.com; connect-src 'self' https://api-m.paypal.com https://api-m.sandbox.paypal.com
- Allowed in logs: order_id, capture_id, invoice_id, custom_id, customer.id, CMID, PayPal-Request-Id, liabilityShift, enrollment_status, authentication_status, PayPal error codes.
- Forbidden in logs: PAN, CVV, full expiry date, cardholder name in free text, access_token, CLIENT_SECRET, full Authorization header.

25. Operational Considerations

25.1 Metrics and Observability

Metric	Granularity	Suggested Alarm
Success rate of POST /v1/oauth2/token	Per minute	< 99% over a 5 min window.
p95 latency of Create Order and Capture Order	Per minute	Exceeds internal SLO.
Rate of liabilityShift !== "POSSIBLE" (when 3DS was activated)	Per hour	Anomaly vs historical baseline.
Non-200 response rate in STC	Per minute	> 5% over a 5 min window.
Rate of UNPROCESSABLE_ENTITY in Create Order	Per minute	> 0.5% indicates a bug in breakdown construction.
Fraudnet inventory injected in the DOM	Per session	More than 2 injected scripts → bug.

25.2 Fault Tolerance and Promotion to Live

Service	Strategy
POST /v1/oauth2/token	Retry once after 401/5xx; if it persists, abort and alert.
POST /v2/checkout/orders	Retry with the same PayPal-Request-Id on 5xx or transient network errors.
PUT /v1/risk/transaction-contexts/... (STC)	Do not retry. Log the failure and continue checkout.
https://c.paypal.com/da/r/fb.js (Fraudnet)	If the script does not load, continue checkout (graceful degradation, increased risk).

Item	Change for production
PAYPAL_API_BASE	https://api-m.sandbox.paypal.com → https://api-m.paypal.com
PAYPAL_CLIENT_ID / PAYPAL_CLIENT_SECRET	Live app credentials, not Sandbox.
PAYPAL_MERCHANT_ID	Merchant ID of the Live account.

Item	Change for production
application_context.return_url / cancel_url	Real URLs in the merchant's production domain.
STC additional_data	Data from the real authenticated user, not test values.

Part V — Validation and Governance

26. Testing Strategy and Sandbox Cards

26.1 Common Sandbox Rules

- Expiration date: any future date.
- CVV: any 3-digit value (4 digits for Amex).
- Cardholder name: free text, except for 3DS where a specific rule applies (see §26.3).

26.2 Cards for Vault and Tokenization

Brand	Number	Notes
Visa (Bancomer)	4772129056533503	Credit card issued by Bancomer (Mexico).
Mastercard	5288775404117508	Credit card.
Amex	376680816376961	4-digit CVV.

26.3 Cards for 3DS Risk Initiated

 To activate 3DS in Sandbox, two simultaneous conditions are required: (1) Card Holder Name = 3dsuser (literal, exact, no extra spaces). (2) Card number from the tables in §26.3.1 or §26.3.2. If either condition is missing, PayPal's risk engine does not trigger 3DS and the transaction processes as a normal purchase.

26.3.1 Frictionless Flows (No User Modal)

Case	Brand	liabilityShift	enrollment	auth	Capture?	Number
Successful frictionless	Mastercard	POSSIBLE	Y	Y	YES	5445492013842209
Successful frictionless	Visa	POSSIBLE	Y	Y	YES	4401331018783148
Failed frictionless	Mastercard	N	Y	N	NO	5445492022569124
Failed frictionless	Visa	N	Y	N	NO	4401331026683975

Case	Brand	liabilityShift	enrollment	auth	Capture?	Number
Attempts Stand-In	Mastercard	POSSIBLE	Y	A	YES	5445492038663051
Attempts Stand-In	Visa	POSSIBLE	Y	A	YES	4401331039804212
Auth unavailable (issuer)	Mastercard	N	Y	U	NO	5445492048677687
Auth unavailable (issuer)	Visa	N	Y	U	NO	4401331042569984
Auth rejected by issuer	Mastercard	N	Y	R	NO	5445492055763685
Auth rejected by issuer	Visa	N	Y	R	NO	4401331055071902
Auth unavailable at lookup	Mastercard	POSSIBLE	Y	A	YES	5445492061636883
Auth unavailable at lookup	Visa	POSSIBLE	Y	A	YES	4401331066329091

26.3.2 Step-Up Flows (With User Modal)

Case	Brand	liabilityShift	enrollment	auth	Capture?	Number
Successful Step-Up	Mastercard	POSSIBLE	Y	Y	YES	5445492100342725
Successful Step-Up	Visa	POSSIBLE	Y	Y	YES	4401331109711123
Failed Step-Up	Mastercard	N	Y	N	NO	5445492119435767
Failed Step-Up	Visa	N	Y	N	NO	4401331117299038
Step-Up unavailable	Mastercard	N	Y	U	NO	5445492122987739
Step-Up unavailable	Visa	N	Y	U	NO	4401331128022452

26.4 Value Dictionary

26.4.1 liabilityShift (in onApprove)

Value	Action
undefined	Capture — there was no 3DS challenge.
"POSSIBLE"	Capture — successful 3DS, liability shifted to issuer.
"N"	Do not capture — authentication failed.
"U"	Do not capture — authentication unavailable.

26.4.2 authentication_status

Value	Meaning
Y	Authentication successful and verified.
N	Authentication failed.
A	Attempts — attempt without full guarantee.
U	Unavailable — bank/system could not complete.
R	Explicitly rejected by issuer.

26.4.3 enrollment_status

Value	Meaning
Y	Card enrolled in 3DS.
N	Not enrolled.
U	Enrollment cannot be determined.

26.5 Minimum Test Cases (TC-01 to TC-14)

ID	Case	Expected Result	Satisfies RF
TC-01	New-card payment, no 3DS, no Vault	Successful capture, token does NOT appear in Vault.	RF-01, RF-02
TC-02	New-card payment, successful 3DS Frictionless	liabilityShift = "POSSIBLE", successful capture.	RF-03
TC-03	New-card payment, successful 3DS Step-Up	Modal appears, capture succeeds after approval.	RF-03
TC-04	New-card payment, failed 3DS Step-Up	Modal appears, NO capture, UI shows clear error.	RF-03, RF-10
TC-05	Vault-with-Purchase	Successful capture, token appears in GET /v3/vault/payment-tokens.	RF-04
TC-06	Saved-card payment (token)	Successful capture or auto-capture, depending on status.	RF-05
TC-07	Token deletion	DELETE returns 204, token disappears from the list.	RF-06
TC-08	3 interest-free Installments payment with new card	Selector shows "3 interest-free months", successful capture.	RF-07
TC-09	6 IC2B payment with new card	Selector shows fee, successful capture with fee_reference_id.	RF-07
TC-10	6 interest-free Installments payment with saved card	calculated-financing-options returns the option, successful capture.	RF-05, RF-07
TC-11	STC returns 400	Checkout is not interrupted; order is created correctly.	RF-08
TC-12	DOM inspection on initialization	Exactly 2 Fraudnet scripts (JSON config + fb.js).	RF-09
TC-13	Retry in the same session after failure	Same CMID, same Fraudnet load.	RF-12
TC-14	BOPIS (if applicable)	shipping.name.full_name with S2S prefix.	RF-11

27. REST vs SDK Naming Conventions

Inconsistency between snake_case (REST) and camelCase (SDK) is the most common source of bugs in Installments and Vault.

Concept	REST API (snake_case)	JavaScript SDK (camelCase)
Financing interval duration	interval_duration	intervalDuration
Option fee reference	fee_reference_id	feeReferenceId
Financing country	financing_country_code	financingCountryCode
Billing country	billing.country_code (object)	billingCountryCode
Currency code	currency_code	currencyCode
Include buyer installments	(not applicable in REST)	includeBuyerInstallments
3DS result in callback	(not applicable in REST)	liabilityShift
Order identifier	id (in response)	orderId (in callbacks)

💡 Mental rule: If the data goes into or comes out of a REST endpoint, it is snake_case. If you pass it to a paypal.* object method or receive it in a callback, it is camelCase.

28. Assumptions, Dependencies, and Constraints

28.1 Assumptions

ID	Assumption
A-01	The merchant has a backend under its control where it can protect PAYPAL_CLIENT_SECRET, PAYPAL_MERCHANT_ID, and issue the access_token.
A-02	The merchant has a user authentication system that produces a stable and deterministic customer.id per buyer.
A-03	The merchant has a cart service that produces a mathematically consistent breakdown.
A-04	The target market supports the configured currency and the applicable Installments/IC2B modes.
A-05	The buyer uses a modern browser with iframe, ES2017+, and CSP support. Degradation for legacy browsers is handled through cardField.isEligible().
A-06	The PayPal account has the commercial enablements listed in §10.1 activated.

28.2 External Dependencies

ID	Dependency	Type
D-01	PayPal REST API (api-m.paypal.com, api-m.sandbox.paypal.com).	Critical — blocking.
D-02	PayPal JavaScript SDK (https://www.paypal.com/sdk/js).	Critical — blocking.
D-03	Card Fields base stylesheet.	Recommended.
D-04	Fraudnet script (https://c.paypal.com/da/r/fb.js).	Recommended — graceful degradation if it does not load.
D-05	OAuth2 endpoint (/v1/oauth2/token).	Critical — without it there is no access_token or id_token.
D-06	STC endpoint (/v1/risk/transaction-contexts).	Recommended — non-blocking.

28.3 Constraints

ID	Constraint
R-01	3DS Risk Initiated only operates with accounts that have Chargeback Protection (CBP) enabled. Accounts with Fraud Protection (FP) require Merchant Initiated mode, outside the scope of this SDD.
R-02	The customer.id in Vault must be stable; changing it invalidates the association with previously saved tokens.
R-03	The CMID is unique per checkout session and must not be reused across buyers or distinct sessions.
R-04	PayPal-Request-Id must be idempotent: same UUID for Create Order and its associated Capture; new one per transaction.
R-05	Fraudnet scripts must be injected in the correct order: JSON configuration first, then fb.js.
R-06	The SDK data-client-metadata-id must exactly match Fraudnet's "f", the STC URL, and the Create/Capture Order header.

29. Risks and Mitigations

ID	Risk	Prob.	Impact	Mitigation
RG-01	Exposure of PAYPAL_CLIENT_SECRET through accidental commit to the repository.	Medium	Critical	Pre-commit hooks that detect secret patterns; secrets manager; mandatory security review. Immediate rotation plan if exposure is detected.
RG-02	Exposure of access_token to the frontend due to a bug in GET /api/token.	Low	Critical	Automated tests that validate the response does not contain access_token; mandatory code review.
RG-03	Multiple Fraudnet injection due to a loading bug.	Medium	Medium	Module-level boolean flag (§13.2); DOM inspection test in QA.
RG-04	CMID desynchronization across Fraudnet, SDK, STC, and headers.	Medium	High	Generate the CMID in a single place and propagate it by reference; regex validation (§12.3).
RG-05	Capturing payments with unfavorable liabilityShift because of a bug in onApprove.	Low	Critical	Automated tests covering the §20.3 matrix; operational alarm on capture rate with liabilityShift !== "POSSIBLE".
RG-06	Duplicate order because of retry without idempotency.	Medium	High	Consistent PayPal-Request-Id across retries (§17.1, §25.2).
RG-07	Mixing snake_case/camelCase in Installments → UNPROCESSABLE_ENTITY.	High	Medium	Conventions table (§27); code review; specific tests for interest-free Installments and IC2B.
RG-08	Loading credentials from the wrong environment (Sandbox in Live or vice versa).	Low	Critical	Validate CLIENT_ID prefix and PAYPAL_API_BASE when the backend starts; alert if inconsistent.
RG-09	PayPal SDK or Fraudnet outage due to external incident.	Low	High	Active monitoring of PayPal domain availability; degradation message to the user; controlled retry.
RG-10	Checkout blocked by STC error (violation of NFR-05).	Low	Critical	STC implemented as non-blocking with try/catch and guaranteed 200 response to the frontend (§14.5.1). QA test: TC-11.

ID	Risk	Prob.	Impact	Mitigation
RG-11	Overly restrictive CSP blocks Fraudnet or SDK.	Medium	High	CSP tested in staging; detection fallback and clear user message if scripts do not load.
RG-12	Demo hardcoded data persists in production (customer.id, STC additional_data, breakdown).	Medium	High	Mandatory code review; §30 checklist; search for suspicious literal strings before each release.

30. Acceptance Criteria and Pre-Production Checklist

Before promoting the solution to Live, all the following criteria must be met.

30.1 Security

- ☐ PAYPAL_CLIENT_SECRET resides only in backend environment variables; it is never included in versioned source code or sent to the browser.
- ☐ Configuration files with credentials (.env, equivalents) are excluded from version control.
- ☐ The backend exposes only clientId and idToken to the frontend; never access_token.
- ☐ Logs do not record PAN, CVV, access_token, or Authorization headers.
- ☐ Content Security Policy allows <https://www.paypal.com>, <https://www.paypalobjects.com>, and <https://c.paypal.com>.
- ☐ TLS 1.2+ enabled in production.

30.2 Configuration

- ☐ PAYPAL_API_BASE points to <https://api-m.paypal.com> in Live.
- ☐ PAYPAL_CLIENT_ID, PAYPAL_CLIENT_SECRET, and PAYPAL_MERCHANT_ID correspond to the Live environment.
- ☐ application_context.return_url and cancel_url are real and accessible URLs in the merchant domain.
- ☐ The SDK data-client-metadata-id attribute uses the dynamically generated CMID, not a literal.
- ☐ The SDK components parameter includes only card-fields.

30.3 Functional

- ☐ customer.id comes from the authenticated user in the merchant platform, not from a literal.
- ☐ breakdown and items reflect the real cart state and comply with the rules in §17.4.
- ☐ PayPal-Request-Id: the same UUID in Create Order and its associated Capture; one new value per transaction.
- ☐ Error handling implemented in onError and onInstallmentsError.
- ☐ The UI explicitly handles liabilityShift !== "POSSIBLE" when present.
- ☐ payer, shipping, and purchase_units data come from checkout state, not placeholders.

30.4 Risk

- ☐ CMID generated once per checkout session.
- ☐ Fraudnet injected once (field "f" with the dynamic CMID).
- ☐ Fraudnet "s" field uses format <SHORT_NAME>_<MERCHANT_ID>_ACDC.
- ☐ STC is called before each Create Order (new and saved card).

- ☐ STC additional_data comes from the buyer's authenticated session.
- ☐ STC does not block checkout: errors are logged and the transaction continues.
- ☐ PayPal-Client-Metadata-Id header present in Create Order and Capture Order, with the same CMID.

30.5 Testing

- ☐ Test-case matrix TC-01 through TC-14 executed and green in Sandbox.
- ☐ DOM inspection confirms exactly two Fraudnet scripts (config + fb.js).
- ☐ STC with PUT /v1/risk/transaction-contexts/... responds 200 before each Create Order.

30.6 Operations

- ☐ Metrics from §25.1 instrumented and connected to the observability system.
- ☐ Alarms configured for the suggested thresholds.
- ☐ Incident runbook documented: OAuth failure, Create Order failure, liabilityShift anomaly.
- ☐ CLIENT_SECRET rotation plan documented and tested in Sandbox.

Appendix A — Troubleshooting Common Errors

Symptom	Most Likely Cause	How to Diagnose
401 Unauthorized in /v1/oauth2/token	CLIENT_ID / CLIENT_SECRET copied incorrectly or crossed environment (Sandbox vs Live).	Verify that PAYPAL_API_BASE and credentials correspond to the same environment.
422 UNPROCESSABLE_ENTITY in Create Order	breakdown does not reconcile with amount.value and/or item sum.	Apply the §17.4 rules manually; PayPal returns the specific inconsistent field in the response.
422 UNPROCESSABLE_ENTITY with IC2B installments	Missing fee_reference_id.	In IC2B (total_consumer_fee.value > 0), fee_reference_id is mandatory in submit (camelCase) or in attributes.installments (snake_case).
Interest-free Installments and IC2B are mixed in the UI	Only term is checked, not total_consumer_fee.	Use parseFloat(opt.total_consumer_fee.value) === 0 to classify.
liabilityShift === "N" and capture is attempted	onApprove does not validate the 3DS result.	Implement the decision table in §20.3.
Saved token does not appear when listing	Different customer.id between Vault-with-Purchase and query.	customer.id must be the same stable buyer identifier at both points.
STC returns 400	Incorrect field type in additional_data.	highrisk_txn_flag must be a number (0 or 1), not a string. sender_create_date must use one of the allowed formats (§14.3).
STC returns 401	Incorrect MERCHANT_ID, expired access_token, or missing permission.	Verify PAYPAL_MERCHANT_ID. Refresh token. Validate enablement with the Integration Engineer.
Fraudnet is injected multiple times	Missing fraudnetLoaded flag.	Implement the module-level boolean pattern from §13.2.
data-client-metadata-id appears empty	SDK loaded before generating the CMID.	Guarantee strict order: §12 → §13 → §11 → §15.
Works in Postman but fails in the browser	Frontend is trying to call api-m.paypal.com directly.	Frontend always calls the backend /api/*; never the PayPal REST API directly.

Symptom	Most Likely Cause	How to Diagnose
Card Fields does not render	cardField.isEligible() returns false or the SDK did not load.	Validate isEligible() before .render(). Check browser console and Network tab.
3DS never activates in Sandbox	Missing 3dsuser name or card not listed.	Satisfy both conditions in §26.3 simultaneously.

Appendix B — Limitations and Future Work

B.1 Known Limitations

- 3DS Merchant Initiated is not covered. Merchants with Fraud Protection (FP) instead of Chargeback Protection (CBP) require a different solution based on `payment_source.card.attributes.verification`.
- The test cards and behaviors described in §26 are those published by PayPal Sandbox at the time this SDD was written. PayPal may update the test set; consult the current official documentation if discrepancies arise.
- Sandbox cards work only in the Sandbox domain. Calling them against Live produces a rejection from the real issuing bank.
- STC industry packs (Travel, OTAs, Gaming, Financial Services, etc.) require additional fields not included in the generic Retail set in §14.2. Request the corresponding pack from the assigned Integration Engineer.
- The REST vs SDK divergence (`snake_case` vs `camelCase`) may change between SDK versions. The loaded SDK version (<https://www.paypal.com/sdk/js>) is the source of truth for callback names.
- The CMID must be unique per checkout session. Do not reuse CMIDs across buyers or across distinct sessions for the same buyer.
- This solution describes the merchant server-to-server integration. Webhooks, refunds, settlement, and disputes require a complementary SDD.

B.2 Suggested Future Work

Initiative	Description
Webhooks	Complementary SDD for subscribing to and processing asynchronous events (<code>PAYMENT.CAPTURE.COMPLETED</code> , <code>PAYMENT.CAPTURE.DENIED</code> , <code>CUSTOMER.DISPUTE.CREATED</code> , etc.).
Refunds	Back-office flow design for partial and full refunds (<code>POST /v2/payments/captures/{id}/refund</code>).
Financial reconciliation	Integration with PayPal reports and settlement for accounting reconciliation.
Additional APMs	Extension of the solution to include Pay Later, Venmo, or local wallets.
Subscriptions	If the business model evolves toward recurrence managed by PayPal Billing Plans.
Industry pack	Adoption of the extended STC <code>additional_data</code> set corresponding to the merchant vertical.

Solution Design Document for the PayPal Advanced Credit and Debit Card (ACDC) integration with Card Fields, 3DS Risk Initiated, Installments (IC2B), Vault, Fraudnet, and Set Transaction Context (STC).